项目技术全解析

> 本文档从代码级别详细拆解整个系统的架构、模块、数据流、安全机制、实验体系和工程细节。
> 面向需要深入理解系统实现的读者（如答辩老师追问、论文审稿、接手开发）。
> 最后更新：2026-03-19

目录

1. 系统全局视图
2. 模型层：切片与导出
3. Worker 层：推理与证明服务
4. Master 层：调度与验证
5. 工具层：EZKL 封装与哈希
6. 三层验证机制逐层拆解
7. Proof-Bound Output 机制详解
8. 选择性验证与 Edge-Cover 策略
9. 可见性模式与隐私设计
10. 攻击模型与故障注入
11. 实验体系全景
12. 测试体系
13. 文件与产物管理
14. 技术栈与依赖
15. 已知限制与边界

一、系统全局视图

1.1 一句话定位

一个将 zkML（零知识机器学习）从单机推理扩展到多节点流水线的研究原型系统。目标是**可验证推理（Verifiable Inference）**，而非隐私推理（Private Inference）。

1.2 系统拓扑

```
                ┌──────────────────────────┐
                │      Master              │
                │   (master.py)            │
                │                          │
                │ · 调度推理请求            │
                │ · 执行三层校验            │
                │ · 随机挑战 light          │
                │ · proof-bound            │
                │   output 验证            │
                └──────────┬───────────────┘
                           │ HTTP POST
        ┌──────────────────┼──────────────────┐
        │                  │                  │
   ┌────▼─────┐       ┌────▼─────┐       ┌────▼─────┐
   │ Worker 1 │       │ Worker 2 │       │ Worker N │
   │ (FastAPI)│       │ (FastAPI)│       │ (FastAPI)│
   │          │       │          │       │          │
   │slice_1.onnx│     │slice_2.onnx│     │slice_N.onnx│
   │+ EZKL 电路│      │+ EZKL 电路│      │+ EZKL 电路│
   └──────────┘       └──────────┘       └──────────┘
```

1.3 数据流（一次完整推理）

```
用户输入 x (float list)
    │
    ▼
Master 发送 x 给 Worker 1
    Worker 1: y1 = f1(x), 可能生成 proof_1
    │
    ▼
Master 验证 Worker 1 的返回，发送 y1 给 Worker 2
```

```
    Worker 2: y2 = f2(y1), 可能生成 proof_2
    |
    ▼
... 依次传递 ...
    |
    ▼
Master 收到 yN = fN(...f2(f1(x)))
    执行最终汇总与验证报告
    |
    ▼
输出最终结果 + 完整验证报告
```

1.4 代码结构一览

```
C:\ZKP\
├── distributed/
|   ├── master.py          # Master 调度 + 三层校验 + 随机挑战
|   └── worker.py          # Worker FastAPI 服务
|
├── common/
|   └── utils.py           # EZKL 初始化/prove/verify + 哈希 + IO
|
├── models/
|   ├── configurable_model.py  # 模型定义 + 按层切分 + ONNX 导出
|   ├── slice_*.onnx           # 导出的各切片 ONNX 文件
|   ├── slice_*_cal.json       # 校准数据
|   └── slice_*_input.json     # 测试输入
|
├── scripts/
|   ├── run_stage2.py              # 主系统功能验证
|   ├── run_experiments.py         # 基础性能实验
|   ├── run_advanced_experiments.py  # 选择性验证 + 攻击实验
|   ├── run_p2_experiment.py       # 可见性模式对比
```

```
│   ├── run_p4_p6_experiment.py    # 补充评估
│   ├── gen_pub_figures.py        # 图表生成
│   └── gen_midterm_ppt.py        # PPT 生成
│
├── tests/
│   └── test_core_semantics.py  # 18 项语义回归测试
│
├── metrics/                    # 实验结果 JSON
├── figures/                    # 生成的图表 PNG
└── artifacts/                  # EZKL 产物（proof, witness, srs 等）
```

二、模型层：切片与导出

2.1 模型定义（configurable_model.py）

模型是一个可配置深度的全连接网络（`ConfigurableModel`）：

```
class ConfigurableModel(nn.Module):
    def __init__(self, input_dim=8, hidden_dim=8, output_dim=4, num_layers=8):
        # 结构: Input(8) → [Linear(8,8) → ReLU] × 7 → Linear(8,4)
```

- 默认 8 层，480 个参数
- 输入维度 8，隐藏维度 8，输出维度 4
- 使用 ReLU 激活函数

2.2 切片逻辑

`split_and_export()` 函数将模型层顺序均匀分组：

- 8 层切 2 片 → 每片 4 层
- 8 层切 4 片 → 每片 2 层
- 8 层切 8 片 → 每片 1 层

每个切片被封装为一个 `SliceModel`，其 `forward()` 方法只执行自己负责的那几层。

2.3 导出流程

每个切片经历以下导出过程：

1. 用 `torch.onnx.export()` 导出为 ONNX 文件（如 `slice_1.onnx`）
2. 生成校准数据文件（`slice_1_cal.json`）：包含若干组输入输出样本，供 EZKL 校准量化参数
3. 生成测试输入文件（`slice_1_input.json`）：前一个切片的输出就是下一个切片的输入
4. 记录中间张量（`intermediates`）：用于调试和 P4 一致性验证

三、Worker 层：推理与证明服务

3.1 Worker 启动流程

Worker 是一个 FastAPI 应用（`worker.py`），启动时做两件事：

1. **加载 ONNX 模型**：用 ONNXRuntime 创建 `InferenceSession`
2. **初始化 EZKL**：调用 `ezkl_init()` 完成“gen_settings → calibrate → compile → get_srs → setup“的全流程

初始化完成后，Worker 就准备好接受推理请求了。

3.2 三个 API 端点

POST /infer — 完整推理 + ZKP 证明

这是 proof 节点使用的端点。完整流程：

```
收到请求 (input_data, request_id)
    |
    ├── 1. 计算 hash_in = SHA256(input_data)
    ├── 2. ONNXRuntime 执行前向推理 → correct_output
    ├── 3. 计算 hash_out = SHA256(correct_output) ← 始终基于正确结果
    ├── 4. 可选故障注入（仅改 output_data，不改 hash_out）
```

```
    │
    ├── 5. 写入 EZKL 输入文件
    ├── 6. 调用 ezkl_prove() → 生成 witness + proof + verify
    │
    ├── 7. ★ Proof-Bound Output ★
    │      从 proof.pretty_public_inputs.rescaled_outputs 提取输出
    │      用 proof 绑定的输出替代 ONNXRuntime 的输出
    │      重新计算 hash_out = SHA256(proof_output)
    │
    └── 8. 返回 InferResponse
```

关键点：步骤 7 是 proof-bound output 的核心。Worker 返回的 `output_data` 不是自己算的，而是从证明中提取的。这意味着即使 Worker 在步骤 4 注入了故障，步骤 7 会用证明里的正确值覆盖篡改值。

POST /infer_light — 轻量推理（无证明）

这是 light 节点使用的端点。只做推理和哈希，不生成 proof：

```
收到请求 (input_data, request_id)
    │
    ├── 1-4. 同 /infer 的前四步（推理 + 可选故障注入）
    │
    ├── 5. 缓存请求数据（供 /re_prove 使用）
    │      request_cache[request_id] = {input_data, hash_in, hash_out, output_data}
    │      缓存上限 100 条，LRU 淘汰
    │
    └── 6. 返回 InferResponse（proof=None, verified=None）
```

关键点：light 节点没有 proof-bound output 的保护。如果被恶意攻击，篡改的输出可以直接传出去。安全依赖 L1 哈希检查（但可被绕过）和随机挑战。

POST /re_prove — 随机挑战重验证

这是 Master 对 light 节点发起的“抽查“：

```
收到请求 (request_id)
    |
    ├── 1. 在 request_cache 中查找 request_id
    |       └── 找不到 → 返回 error: challenge_cache_miss
    |
    ├── 2. 取出缓存的 input_data
    ├── 3. 用缓存数据调用 ezkl_prove() 生成证明
    ├── 4. 检查 cache_consistent: 缓存的 hash_out == SHA256(缓存的 output_dat
a)
    |
    └── 5. 返回 {verified, proof, proof_instances, cache_consistent, from_cache}
```

关键设计：re_prove 是**严格模式**。必须提供有效的 request_id 并命中缓存，不允许 fallback 到新输入。这确保挑战是对**历史请求**的可追溯复核。

3.3 请求/响应数据结构

InferRequest:

```
{
    "input_data": [0.1, 0.2, ...],   # flat 浮点数列表
    "request_id": "req-1-1773813035011"  # 可选，不传则自动生成
}
```

InferResponse:

```
{
    "request_id": "req-1-1773813035011",
    "slice_id": 1,
    "output_data": [0.3, -0.1, ...],     # 推理输出（proof 节点为 proof-bound）
    "hash_in": "a1b2c3...",              # SHA256(input_data)
    "hash_out": "d4e5f6...",             # SHA256(output_data)
    "proof": { ... },                    # 完整 proof JSON（light 节点为 null）
```

```
    "verified": true,                    # Worker 本地 verify 结果
    "proof_instances": {                 # ZKP 公开实例
        "processed_inputs": [...],       #  电路输入（Poseidon 哈希或原始量化值）
        "processed_outputs": [...],      #  电路输出
        "rescaled_inputs": [...],        #  反量化后的输入
        "rescaled_outputs": [...]        #  反量化后的输出（proof-bound 的数据源）
    },
    "proof_artifacts": {                 # 产物路径
        "proof_path": "...",
        "witness_path": "...",
        "settings": "...",
        "vk": "...",
        "srs": "..."
    },
    "metrics": {
        "proof_gen_ms": 1234.5,
        "verify_ms": 87.3,
        "forward_ms": 1350.0,
        "peak_rss_mb": 156.2,
        "proof_size_bytes": 77619,
        "witness_size_bytes": 10137,
        "request_id": "req-1-1773813035011"
    },
    "fault_injected": false,
    "proof_mode": "full"                 # "full" 或 "light"
}
```

四、Master 层：调度与验证

4.1 核心函数：run_pipeline()

`master.py` 的 `run_pipeline()` 是整个系统的调度中枢：

```python
def run_pipeline(
    initial_input,     # 第一个切片的输入
    workers,           # Worker 配置列表
    fault_at=None,     # 在哪个 slice_id 注入故障
    fault_type="tamper",
    verify_ratio=1.0,  # 期望验证比例
    verify_strategy="edge_cover",
    seed=None,
)
```

4.2 流水线执行流程

```
1. 调用 _select_verified_slices() 确定哪些切片做 proof
2. 记录 actual_proof_fraction

3. FOR 每个 Worker (顺序执行):
   |
   ├── 确定调用 /infer 还是 /infer_light
   ├── 确定是否注入故障
   ├── HTTP POST 发送请求
   |
   ├── 对 proof 节点:
   |   ├── Master 独立调用 ezkl_verify_proof() 验证 proof
   |   ├── 从 proof.pretty_public_inputs 提取 linking 数据
   |   ├── ★ 提取 rescaled_outputs 做 proof-output 交叉验证
   |   └── 如果 proof-output 不一致 → 记录 output_proof_mismatch
   |
   ├── L1 校验: SHA256(output_data) == hash_out ?
   ├── L2 校验: prev.processed_outputs == curr.processed_inputs ?
   ├── L3 校验: prev.hash_out == curr.hash_in ?
   |
   ├── ★ Proof-Bound Override ★
   |   如果是 proof 节点且提取到 rescaled_outputs:
```

```
    │   → current_input = proof 中的输出（不用 Worker 声称的输出）
    │
    └── 否则: current_input = data["output_data"]

4. 流水线结束后:
    ├── 对 light 节点发起随机挑战 (/re_prove)
    ├── 首节点验证
    └── 汇总所有 metrics 和 findings
```

4.3 Master 独立验证

Master 不信任 Worker 的 `verified` 字段。它拿到 proof 文件后，自己调用 `ezkl_verify_proof()` 重新验证：

```
local_verified = ezkl_verify_proof(proof_path, verify_paths)
# 如果 local_verified != worker_verified → 记录 verify_mismatch
```

4.4 Proof-Output 交叉验证

Master 从 proof 的 `pretty_public_inputs.rescaled_outputs` 中独立提取输出，与 Worker 声称的 `output_data` 对比。如果不一致，说明 Worker 可能篡改了输出但提交了正确的 proof。Master 会：

1. 记录 `output_proof_mismatch` 到 l2_findings
2. 以 proof 中的输出为准传递给下一个 Worker

五、工具层：EZKL 封装与哈希

5.1 ezkl_init() — 一次性初始化

这是整个 ZKP 管线的基础设置，Worker 启动时调用一次：

```
gen_settings(onnx, settings.json, visibility_mode)
```

```
    → 根据 visibility_mode 设置 input/output/param 的可见性
    |
calibrate_settings(cal.json, onnx, settings.json)
    → 用校准数据自动确定量化参数（input_scale, param_scale 等）
    |
compile_circuit(onnx, network.compiled, settings.json)
    → 将 ONNX 模型编译为 Halo2 算术电路
    |
get_srs(settings.json, kzg.srs)
    → 下载或生成结构化参考字符串（KZG commitment 的公共参数）
    |
setup(network.compiled, vk.key, pk.key, kzg.srs)
    → 生成验证密钥 (vk) 和证明密钥 (pk)
```

产物：

- `settings.json`：电路配置（量化参数、可见性、logrows 等）
- `network.compiled`：编译后的 Halo2 电路
- `pk.key` / `vk.key`：证明密钥 / 验证密钥
- `kzg.srs`：KZG 多项式承诺的公共参数

5.2 ezkl_prove() — 证明生成 + 验证

每次推理请求都会调用：

```
gen_witness(input.json, network.compiled, witness.json)
    → 执行电路并生成 witness（包含 processed_inputs/outputs）
    |
prove(witness.json, network.compiled, pk.key, proof.json, kzg.srs)
    → 生成零知识证明（1-5 秒）
    |
verify(proof.json, settings.json, vk.key, kzg.srs)
    → 验证证明有效性（30-140 毫秒）
```

关键输出：

- `proof.json`：包含证明数据和 `pretty_public_inputs`（rescaled_inputs, rescaled_outputs, processed_inputs, processed_outputs）
- `witness.json`：包含电路执行的完整 witness 数据

5.3 哈希工具

```
def sha256_of_list(data_list: list) -> str:
    """对浮点数列表做 json.dumps → encode → SHA-256。"""
    serialized = json.dumps(data_list, sort_keys=True).encode("utf-8")
    return hashlib.sha256(serialized).hexdigest()
```

注意：使用 `sort_keys=True` 确保序列化结果确定性。

六、三层验证机制逐层拆解

6.1 L1：输出完整性检查

```
actual_output_hash = sha256_of_list(data["output_data"])
output_integrity = (data["hash_out"] == actual_output_hash)
```

- **对 proof 节点**：由于 proof-bound output 已将 output_data 绑定为 proof 内的正确值，hash_out 也被重新计算，L1 一定通过
- **对 light 节点**：恶意 Worker 可以同时伪造 output_data 和 hash_out 使两者一致，绕过 L1。所以 L1 对 light 节点的安全价值很低
- **安全定位**：故障检测（fault detection），非对抗安全

6.2 L2：ZKP Proof Linking

```
prev_out = prev_instances.get("processed_outputs")
curr_in = curr_instances.get("processed_inputs")
proof_linked = (prev_out == curr_in)
```

- **仅在两端均为 proof 节点时有效**

- **hashed 模式**下，processed_outputs/inputs 是 Poseidon 哈希（电路内部计算，不可伪造），具有密码学级安全性
- **all_public 模式**下，两个独立编译的电路使用不同的量化参数，同一浮点数据量化为不同 field element → linking 不适用 → Master 检测到不匹配后跳过
- **安全定位**：密码学级跨节点绑定（仅 hashed 模式 + 双端 proof）

6.3 L3：跨节点哈希链

```
if i > 0:
    prev_hash_out = results[-1]["hash_out"]
    curr_hash_in = data["hash_in"]
    chain_ok = (prev_hash_out == curr_hash_in)
```

- 检测数据在节点间传输是否被篡改
- **安全定位**：consistency check，非对抗安全（恶意中间人可以同时修改数据和哈希）

6.4 随机挑战

Master 对 light 节点发起 `/re_prove`：

1. 指定 request_id，要求 Worker 用缓存的原始输入重新生成 proof
2. Worker 必须命中缓存（否则直接返回失败）
3. Master 验证新生成的 proof，并检查 cache_consistent

安全机制：如果 light 节点之前偷偷篡改了输出但缓存了正确的输入，re_prove 会生成基于正确输入的 proof，其 rescaled_outputs 将和 light 阶段声称的 output 不一致，从而暴露篡改。

七、Proof-Bound Output 机制详解

7.1 问题

原始设计中，Worker 有两条独立的数据路径：

- 路径 A：ONNXRuntime 推理 → output_data → 返回给 Master
- 路径 B：EZKL gen_witness + prove → proof → 返回给 Master

两条路径独立，恶意 Worker 可以让路径 B 生成正确的 proof，但路径 A 返回篡改的 output。Master 验证 proof 通过，信了 output，但 output 是假的。

7.2 解决方案

Worker 侧（worker.py /infer 端点）：

```
proof_ppi = result["proof"].get("pretty_public_inputs", {})
rescaled = proof_ppi.get("rescaled_outputs", [])
if rescaled:
    output_data = flatten(rescaled)  # 从 proof 中提取输出
    hash_out = sha256_of_list(output_data)  # 重新计算哈希
```

Master 侧（master.py run_pipeline）：

```
ppi = proof_data.get("pretty_public_inputs", {})
master_rescaled = ppi.get("rescaled_outputs")
if master_rescaled:
    master_output = flatten(master_rescaled)
    if master_output != worker_output:
        # 记录 output_proof_mismatch
    current_input = master_output  # 以 proof 输出为准
```

7.3 安全性分析

- **正常场景**：Worker 和 Master 从同一份 proof 提取输出，天然一致
- **篡改场景**：Worker 试图篡改 output_data，但 proof 里的 rescaled_outputs 是电路计算的正确值。Master 独立提取后会发现不一致，并以 proof 输出为准
- **效果**：proof 节点的篡改被**预防**（prevention），不是检测后报警

八、选择性验证与 Edge-Cover 策略

8.1 _select_verified_slices() 的实现

```
def _select_verified_slices(num_slices, verify_ratio, strategy="edge_cover", max_light_gap=1):
```

三种策略：

edge_cover（推荐）：

1. 首尾必选（slice 1 和 slice N）
2. 从前往后扫描，如果连续 light 节点数超过 `max_light_gap`，强制插入 proof 节点
3. 如果 verify_ratio 允许更多，随机补充

contiguous：首尾必选 + 随机选一段连续 proof 节点

random（旧）：首尾必选 + 中间随机散布

8.2 edge_cover 示例（8 切片，verify_ratio=25%）

```
步骤 1: 首尾必选 → {1, 8}
步骤 2: 扫描，max_light_gap=1
  位置 2: light（gap=1，OK）
  位置 3: 需要插入 proof（gap 将超过 1）→ {1, 3, 8}
  位置 4: light（gap=1，OK）
  位置 5: 需要插入 → {1, 3, 5, 8}
  位置 6: light（gap=1，OK）
  位置 7: 需要插入 → {1, 3, 5, 7, 8}

最终: {1, 3, 5, 7, 8} → actual_proof_fraction = 5/8 = 62.5%
```

用户设了 25%，实际覆盖率 62.5%。这是 edge-cover 约束导致的——它必须保证每对相邻节点至少一端是 proof。

8.3 安全保证

定理：在 edge_cover (g=1) 下，攻击段长度 ℓ 的逃逸概率 ≤ 0.5^⌊ℓ/2⌋

因为最多只有 1 个连续 light 节点，任何攻击段的两端都必然至少有一端是 proof 节点，该 proof 节点的 proof-bound output 会拦截篡改。

九、可见性模式与隐私设计

9.1 三种模式的 EZKL 配置

模式	input_visibility	output_visibility	param_visibility	效果
all_public	public	public	fixed	输入在 proof 公开实例中以明文出现
hashed	hashed	public	hashed	输入以 Poseidon 哈希出现，不可反推
private	private	public	fixed	输入不出现在公开实例中

9.2 对 L2 Linking 的影响

- **hashed 模式**：processed_inputs/outputs 是 Poseidon 哈希（电路内计算），跨节点比对有密码学意义
- **all_public 模式**：两个切片独立编译电路，量化参数不同（input_scale, param_scale），同一浮点数据量化为不同 field element → linking 不匹配 → Master 跳过
- **private 模式**：processed_inputs 为空（private 不暴露），linking 无数据 → Master 跳过

9.3 开销对比

模式	证明时间倍数	proof 大小倍数	原因
all_public	1.00×	1.00×	基准

模式	证明时间倍数	proof 大小倍数	原因
hashed	1.61×	1.18×	Poseidon 哈希电路引入额外约束
private	0.97×	0.94×	不引入额外电路计算

十、攻击模型与故障注入

10.1 当前攻击模型

响应层篡改（Response-Layer Tampering）：

- Worker **确实**按照正确的模型做了推理（ONNXRuntime 正常执行）
- 但在返回结果时**篡改了 output_data**
- hash_out 始终基于正确输出计算（模拟“聪明的攻击者只篡改数据不篡改哈希“的提前检测场景）

10.2 四种故障注入方式

```
if fault_type == "tamper":
    output_data[0] += 999.0          # 第一个元素加 999

elif fault_type == "skip":
    output_data = [0.0] * len(output_data)  # 全部归零

elif fault_type == "random":
    output_data = [random.uniform(-10, 10) for _ in output_data]  # 随机数

elif fault_type == "replay":
    output_data = [0.42] * len(output_data)  # 固定值（模拟重放攻击）
```

10.3 攻击处理结果

攻击位置	proof 节点	light 节点
处理方式	proof-bound 预防	L1 检测 + 随机挑战
安全等级	确定性预防	概率性检测

10.4 不在当前攻击模型内的威胁

- 恶意 prover（在电路层面伪造计算）
- 多节点协同攻击
- 拒绝服务 / 延迟攻击
- 训练后门攻击
- Master 被攻破

十一、实验体系全景

11.1 实验脚本对照

脚本	实验内容	管线类型
`run_stage2.py`	主系统功能验证	完整（Master 完整逻辑）
`run_experiments.py`	基础性能（2/4/8 切片）	简化（仅 L1+L3）
`run_advanced_experiments.py`	P1 选择性验证 + P3 攻击	简化（仅 L1+L3）
`run_p2_experiment.py`	P2 可见性模式对比	专项
`run_p4_p6_experiment.py`	P4 一致性 + P6 完整性机制	专项

11.2 六组实验概要

实验	核心指标	主要结论
基础性能	e2e 时延, proof 时间, verify 时间	proof 占 ~85%
P1 选择性验证	actual_proof_fraction, 开销降低	~36%-42%
P2 可见性模式	各模式证明时间, 产物大小	hashed 1.6×, private ≈ public
P3 攻击处理	4 种攻击 × 2 种 verify_ratio	全部预防
P4 切片一致性	L1/L2 距离, 最大误差, 相对误差	全零 (bit-exact)
P6 完整性机制	三种机制的 proof 时间	Poseidon 1.71×, private 1.03×

11.3 实验结果存储

所有结果以 JSON 格式存储在 `metrics/` 目录：

- `stage3_experiments.json`
- `advanced_experiments.json`
- `p2_visibility_modes.json`
- `p4_p6_results.json`

十二、测试体系

12.1 测试设计思路

`tests/test_core_semantics.py` 包含 18 项**语义回归测试**。这些测试不启动 Worker 或运行 EZKL，而是通过检查代码结构来验证安全语义是否闭合：

- **TestWorkerReProveReturnsProof**：/re_prove 返回中是否包含 proof 字段
- **TestWorkerReProveNoFallback**：/re_prove 是否禁止了 fallback 到新输入
- **TestWorkerReProveCacheMiss**：缓存未命中时是否返回错误而非静默放过
- **TestWorkerRequestIsolation**：request_id 是否全局唯一（含时间戳后缀）
- **TestWorkerCacheFindingsIncluded**：re_prove 返回是否包含 from_cache 和 cache_consistent

- **TestMasterIndependentVerify**：Master 是否独立调用 ezkl_verify_proof
- **TestMasterChainLinkingSource**：L2 linking 数据是否从 proof.pretty_public_inputs 提取
- **TestWorkerProofBoundOutput**：/infer 中是否有 rescaled_outputs 提取逻辑
- **TestMasterProofOutputBinding**：Master 是否独立提取 proof 输出做交叉验证
- … 等 18 项

12.2 运行方式

```
python -m pytest tests/test_core_semantics.py -v
# 或
python -m unittest tests.test_core_semantics -v
```

十三、文件与产物管理

13.1 EZKL 静态产物（Worker 启动时生成一次）

```
artifacts/worker_1/
├── settings.json       # 电路配置
├── network.compiled    # 编译后的 Halo2 电路
├── pk.key              # 证明密钥
├── vk.key              # 验证密钥
└── kzg.srs             # KZG 结构化参考字符串
```

13.2 每次推理请求的产物

```
artifacts/worker_1/
├── input_req-1-1773813035011.json        # 临时输入文件（用完即删）
├── input_req-1-1773813035011_witness.json  # witness
└── input_req-1-1773813035011_proof.json    # proof
```

文件名中包含 request_id，确保请求隔离。

13.3 产物大小参考

产物	典型大小
proof.json	~76 KB
witness.json	~10 KB
settings.json	~1 KB
network.compiled	~200 KB
kzg.srs	~1 MB

十四、技术栈与依赖

组件	版本	作用
Python	3.13.12 (Miniconda)	运行时
EZKL	23.0.5	ZKP 引擎 (Halo2/PLONK/KZG)
PyTorch	2.10.0	模型定义与切片
ONNX	1.20.1 (opset 18)	模型交换格式
ONNXRuntime	1.24.3	推理引擎
FastAPI	0.135.1	Worker HTTP 服务
uvicorn	0.42.0	ASGI 服务器
requests	2.32.5	Master HTTP 客户端
psutil	7.2.2	内存监控
matplotlib	3.10.x	图表生成
python-pptx	1.0.2	PPT 生成

组件	版本	作用
python-docx	1.2.0	Word 文档生成
nest_asyncio	~	Windows 上嵌套事件循环兼容

十五、已知限制与边界

15.1 架构限制

限制	说明	影响
Master 可信	所有验证在 Master 执行	Master 被攻破则全线失效
单机模拟	多进程模拟分布式	未验证网络延迟、分区等真实场景
顺序流水线	Worker 必须按顺序执行	无法并行推理

15.2 安全限制

限制	说明
攻击模型窄	仅覆盖响应层篡改
light 节点弱	依赖外部哈希 + 概率性挑战
无端到端绑定	跨节点中间数据离开 proof 后为明文
L2 linking 受限	仅 hashed 模式有效

15.3 实验限制

限制	说明
模型小	480 参数 FC 网络
简化管线	部分实验未用 Master 完整逻辑

限制	说明
攻击位置	故障节点恰好在 proof 位置
统计弱	大部分实验只跑 1 次（P2 跑了 3 次）

15.4 诚实定位

> 这是一个**研究原型**，验证了“分布式推理 + ZKP“技术路线的可行性。它不是一个可以直接部署的生产系统，也不是一个完整的密码学闭环。它的价值在于：证明了这条路线可以走通，定量展示了性能-安全-隐私三者之间的 tradeoff，并为后续工作提供了可运行的基础设施。